TIP You can configure your `AppDbContext`'s connection string in other ways, such as with the `OnConfiguring` method, but I recommend the method shown here for ASP.NET Core websites.

Now you have a `DbContext`, named `AppDbContext`, registered as a scoped service with the DI container (typical for database-related services), and a data model corresponding to your database. Codewise, you're ready to start using EF Core, but the one thing you *don't* have is a database! In section 12.3 you'll see how you can easily use the .NET CLI to ensure that your database stays up to date with your EF Core data model.

12.3 Managing changes with migrations

In this section you'll learn how to generate SQL statements to keep your database's schema in sync with your application's data model, using migrations. You'll learn how to create an initial migration and use it to create the database. Then you'll update your data model, create a second migration, and use it to update the database schema.

Managing schema changes for databases, such as when you need to add a new table or a new column, is notoriously difficult. Your application code is explicitly tied to a particular version of a database, and you need to make sure that the two are always in sync.

DEFINITION *Schema* refers to how the data is organized in a database, including the tables, columns, and relationships among them.

When you deploy an app, normally you can delete the old code/executable and replace it with the new code. Job done. If you need to roll back a change, delete that new code, and deploy an old version of the app.